Space Complexity of Infinite Turing Machines

Dylan Stephano-Shachter*
Department of Mathematics
University of Michigan
Ann Arbor, Michigan USA

August 13, 2015

Abstract

In this paper we present a new model for space complexity of transfinite computation called the infinite Turing machine (ITM). This model differs from the more frequently studied infinite time Turing machine (ITTM) in that it allows for a tape of size larger than ω. In the paper a basic hierarchy theorem is presented as well as a proof that the space requirements of all ITM computations are countable.

1 Introduction

Recently, there has been more and more interest in transfinite computation. The most common model for transfinite computation is the infinite time Turing machine (ITTM). An ITTM functions the same way as a classical Turing machine except that it can continue computing for any transfinite ordinal number of steps [1]. Lots of work has been done in the theory of ITTMs. It is only recently, however, that people have started to consider infinite amounts of memory. A recent paper [3] has developed a theory of α-tape infinite time Turing machines (α-ITTM). These machines function similarly to ITTMs except that they have a tape of some ordinal length α.

The α-ITTM model is a sound model. I believe, however, that it does not represent a good analog to classical space complexity. The reason for this is that in the α-ITTM model, when the read-write head moves off of the end of the tape, it is immediately placed at the first cell. While this might seem like a reasonable convention, it allows for the machine to find the end of its tape. This is in contrast to classical complexity theory, where there is no special marker telling the machine "stop here". It simply does not compute any further.

2 Infinite Turing Machines

The model we will be using is the infinite Turing machine (ITM). The model of computation has an infinite ordinal tape size and can compute for any ordinal number of steps. It has

*This work was patially supported by NSF grant 1045119 under the supervision of Andreas Blass

a tape alphabet of $\{0,1\}$ and it has a set of states which is always a natural number (the set of all smaller natural numbers). The result of computation at successor ordinal steps is exactly the same as with a classical Turing machine. The result at any limit ordinal step is that the machine state, head position, and contents of each cell take on the *lim inf* of their previous values.

The following definitions are largely based on the definitions in [3].

Definition 2.1. Let a command be any 5-tuple (s, c, c', m, s') such that s and s' are in ω and c, c', and m are in $\{0,1\}$. A command should be interpreted as, if the machine is in state s with tape contents c, write c', move in direction m, and change to state s'.

Definition 2.2. A program is a finite set P of commands with the following conditions.

1. If $(s, c, c', m, s') \in P$, then there is some $(s, d, d', n, t) \in P$ with $c \neq d$.
2. If $(s, c, c', m, s') \in P$ and $(s, c, c'', m', s'') \in P$, then $(c', m, s') = (c'', m', s'')$.
3. The set of all states s such that $(s, c, c', m, s') \in P$ is a natural number denoted by $states(P)$.

Definition 2.3. Let P be a program (coded by p). If α is an ordinal and $x \in {}^{\alpha}2$, then a T_α-*computation*, $\alpha_p(x)$, is a triple $S : \theta \to \omega$, $H : \theta \to \alpha + 1$, $T : \theta \to {}^{\alpha}2$ such that:

1. θ is an ordinal or $\theta =$**Ord**. We call θ the *length* of the computation.
2. We call $S(\theta)$, $H(\theta)$ and $T(\theta)$ respectively the *state*, *head position*, and *tape configuration* at time θ. $S(0) = 2$ and $H(0) = 0$ and $T(0) = x$.
3. If $t < \theta$ and $S(t) = 0$, then $\theta = t + 1$. The computation *halts* if the machine's state is 0, the halt state.
4. If $t < \theta$ and $H(t) \notin \alpha$, then $\theta = t + 1$ and $S(\theta) = 1$. If the machine tries to go beyond the end of the tape, it enters state 1, the fail state.
5. If $t < \theta$, $S(t) \in states(P) - \{0,1\}$, and $H(t) \in \alpha$, then $t + 1 < \theta$. Let (s, c, c', m, s') be the unique command in P such that $S(t) = s$ and $T(t)_{H(t)} = c$. Then:

   a) $S(t+1) = s'$

   b)
   $$H(t+1) = \begin{cases} H(t) + 1 & \text{if } m = 1 \\ H(t) - 1 & \text{if } m = 0 \text{ and } H(t) \text{ is a successor ordinal} \\ 0 & \text{otherwise} \end{cases}$$

   c)
   $$T(t+1)_\xi = \begin{cases} c' & \text{if } \xi = H(t) \\ T(t)_\xi & \text{otherwise} \end{cases}$$

6. At limit ordinal times $t < \theta$, the machine takes on inferior limits of previous values:

$a)$ $S(t) = \liminf_{r \to t} S(r)$

$b)$ $H(t) = \liminf_{s \to t} H(s)$

$c)$ For all $\xi \in \alpha$, $T(t)_\xi = \liminf_{r \to t} T(r)_\xi$.

If a program p run on T_α with input $x \in {}^\alpha 2$ halts with output $y \in {}^\alpha 2$, we write $\alpha_p(x) \downarrow y$. If the program fails we write $\alpha_p(x)*$. If the program enters a never-ending loop we write $\alpha_p(x) \downarrow$. We say a function f is $T_\alpha - computable$ if there exists a program p such that for any input $x \in dom(f)$, we have $\alpha_p(x) \downarrow f(x)$.

Definition 2.4. We write $T_\beta \succeq T_\alpha$ to mean for all functions $f : {}^{min(\alpha,\beta)}2 \to {}^{min(\alpha,\beta)}2$, if there is a program p such that $\forall x\, \alpha_p(x) \downarrow f(x)$, then there is a program q such that $\forall x\, \beta_q(x) \downarrow f(x)$.

Lemma 2.5. *If $\beta \geq \alpha$, then $T_\beta \succeq T_\alpha$*

Proof. This is immediate from the definition of ITM computation. □

Definition 2.6. We write $T_\beta \succ T_\alpha$ to mean $T_\beta \succeq T_\alpha$ and there exists a function $f : {}^{min(\alpha,\beta)}2 \to {}^{min(\alpha,\beta)}2$ such that f is T_β-computable but not T_α-computable.

3 Strength Comparison of Tape Sizes

The next logical step is to determine if there is any tape size which gives you more computing power than ω. It turns out that the ordinal $\gamma(\omega)$ is a large enough countable ordinal.

Definition 3.1. An ordinal δ is called $T_\alpha - clockable$ if there exists a program p such that $\alpha_p(0) \downarrow$ after exactly δ many steps. Let $\gamma(\alpha)$ denote the supremum of T_α-clockable ordinals.

Two useful tools in analyzing ITMs are the halting sets. The strong halting set H^α is the set $\{\langle p, x\rangle \in \omega \times {}^\omega 2 | \alpha_p(x) \downarrow\}$. The weak halting set h^α is the set $\{p \in \omega | \alpha_p(0) \downarrow\}$. Both sets have the property that they are not decidable by any T_α-machine.

Theorem 3.2. *For all cardinals κ, computation on any device T_α such that $|\alpha| = \kappa$ will either halt, fail, or enter an infinite loop within a number of steps of cardinality κ.*

Proof. The proof of this theorem has been omitted as it is nearly identical to the proof of Theorem 1.1 from [1]. □

The above theorem tells us that $\gamma(\alpha)$ has cardinality $|\alpha|$. This will allow us to prove the following theorem:

Theorem 3.3. *$T_{\gamma(\omega)} \succ T_\omega$. In general, if α is T_β-clockable and $\beta \geq \gamma(\alpha)$, then $T_\beta \succ T_\alpha$.*

Proof. We know that $T_{\gamma(\omega)} \succeq T_\omega$ due to the fact that $\gamma(\omega) \geq \omega$. What remains is to show that $T_{\gamma(\omega)}$ can decide the weak halting set, h^ω. In order to solve this problem we will run 2 $T_{\gamma(\omega)}$ machines. We will call them machine a and machine b. Machine b will simulate machine a on one tape while compting on another. On input p, machine a will run the computation $\omega_p(0)$. Machine b will continue to run the following loop until it halts.

1. Copy the state, head position, and tape (the configuration) of machine a to the first ω size block of cells which are still empty on machine 2's tape.

2. Go through each configuration written on the tape and check if it matches the configuration of machine a. If a match is found, halt and return 0.

3. Compute one step on machine 1. If machine 1 halts, return 1. If machine 1 fails, return 0.

If $\omega_p(0)$ halts or fails, this algorithm will halt and return the correct answer due to step 3 of the loop. If $\omega_p(0)$ enters an endless loop, then the algorithm will halt on step 2 of the loop after using at most $\gamma(\omega)$ cells. Thus this algorithm decides the weak halting set h^ω

In the more general case, to show that $T_\beta \succ T_\alpha$, simply use the same algorithm except run two T_β machines in parallel. The first of them simulates a T_α computation. The fact that α is T_β-clockable ensures that T_α can be properly simulated. The fact that $\beta \geq \gamma(\alpha)$ ensures that the machine will halt before it reaches the end of its tape. □

4 Limits of Infinite Computation

When looking specifically at inputs of size ω (or any other constant input length), the increase in power discussed in the previous section does not continue forever. In other words there is some γ_1 for which T_{γ_1} can compute anything computable by any infinite Turing machine. Thus for the rest of this section we will assume be default that inputs are of size ω.

Theorem 4.1. *There exists an ordinal denoted δ such that for any ordinal α, $T_\delta \succeq T_\alpha$. Let γ_1 denote the smallest such δ.*

Proof. Let $X = \omega \times {}^\omega 2$ represent the set of all programs and inputs. Consider the set $Y = \{\langle p, x\rangle \in X | \alpha_p(x) \downarrow \text{ for some } \alpha\}$. For each element $\langle p, x\rangle \in Y$ let $\alpha_{p,x}$ denote the supremum of all tape cells used in the computation $\alpha_p(x)$ where α is any ordinal sufficiently large that the computation will not fail. Let δ be the supremum of all the $\alpha_{p,x}$ for all $\langle p, x\rangle \in Y$. It is easy to show that δ satisfies the theorem.

Assume there exists some ξ such that $T_\xi \succ T_\delta$. Then there exists some program p and some input x such that $\xi_p(x) \downarrow$ but $\delta_p(x)*$. But that would mean that $\alpha_{p,x} > \delta$. This is a contradiction since δ is the supremum of all $\alpha_{p,x}$. □

The ordinal γ_1 is a very important ordinal because it is the maximum size tape that will ever be used in a halting ITM computation (Given inputs in ${}^\omega 2$). It will be important to find out whether γ_1 is countable or not. If it is countable that will allow us to restrict all attention to countable tape sizes.

Definition 4.2. An ordinal α is called *clockable* if there exists an ordinal β such that α is T_β-clockable. Let γ_0 denote the supremum of all clockable ordinals (There are only countably many).

Theorem 4.3. *An ordinal α is clockable if and only if it is $\alpha + 1$-clockable.*

Proof. By definition, we have already have that if α is $\alpha+1$-clockable, then it is clockable. If α is clockable, then there exists some ordinal β and some $p \in \omega$ such that $\beta_p(0)\downarrow$ (after α steps). If $\alpha+1 \geq \beta$, then $T_{\alpha+1} \succeq T_\beta$ and so $(\alpha+1)_p(0)\downarrow$ (after α steps). If $\alpha+1 < \beta$, we know that $p(0)$ takes only α steps so only $\alpha+1$ tape size is necessary for $p(0)$ to halt. So $\alpha+1_p(0)\downarrow$ (after α steps). □

The fact that we can use $\alpha+1$-clockable as an alternative definition of clockable is very important as it allows for the notion to be defined by a formula that does not quantify over all ordinals. This is very important to the following lemma which will help to show that all clockable ordinals are countable.

Lemma 4.4. *The formula "$(\beta+1)_p(0)\downarrow$ (after β steps)" is Δ_1*

Proof. The formula is equivalent to, $\exists S : \theta \to \omega\, \exists H : \theta \to \beta+2\, \exists T : \theta \to^{\beta+1} 2\, (\phi_A \wedge \phi_B \wedge \phi_C \wedge \phi_D \wedge \phi_E \wedge \theta = \beta)$ and it is equivalent to $\forall S : \theta \to \omega\, \forall H : \theta \to \beta+2\, \forall T : \theta \to^{\beta+1} 2\, (\phi_A \wedge \phi_B \wedge \phi_C \wedge \phi_D \wedge \phi_E \to \theta = \beta)$, where the ϕ's represent:

1. ϕ_A is $S(0) = 2 \wedge H(0) = 0 \wedge T(0) = \langle 0, 0, ...\rangle$
2. ϕ_B is $(t < \theta \wedge S(t) = 0) \to \theta = t+1$
3. ϕ_C is $(t < \theta \wedge H(t) \notin \beta+1) \to \theta = t+1 \wedge S(\theta) = 1$
4. ϕ_D is $(t < \theta \wedge S(t) \in states(P) \wedge H(t) \in \beta+1) \to \exists y \in P\, [y = (s, c, c', m, s') \wedge S(t+1) = s' \wedge (m = 1 \to H(t+1) = H(t)+1) \wedge (m = 0 \wedge \neg lim(H(t)) \to H(t+1) = H(t)-1) \wedge (m = 0 \wedge lim(H(t)) \to H(t+1) = 0) \wedge (\forall \xi \in \beta+2 (\xi = H(t) \to T(t+1)_\xi = c') \wedge (\xi \neq H(t) \to T(t+1)_\xi = T(t)_\xi))]$
5. ϕ_E is $(t < \theta \wedge lim(t)) \to (S(t) = \lim_{r\to t} \inf S(r) \wedge H(t) = \lim_{s\to t} \inf H(s) \wedge \forall \xi \in \alpha+1, T(t)_\xi = \lim_{r\to t} \inf T(r)_\xi)$

Note that in the above formulas $lim(t)$ refers to the formula "t is a limit ordinal" which is Δ_1. All the formulas above are known to be Δ_1 statements. The only part which may need proof is that $(S(t) = \lim_{r\to t} \inf S(r)$ is Δ_1. This is clear from its more expanded form $(\forall r < t\, \exists \gamma < t\, r < \gamma \wedge S(\gamma) = S(t)) \wedge (\forall \delta < S(t) \neg ((\forall r < t\, \exists \gamma < t\, r < \gamma \wedge S(\gamma) = \delta))$

Since the statement is equivalent to a formula with only existential quantifiers and Δ_1 formulas as well as a formula with only universal quantifiers and Δ_1 formulas, it is Δ_1.

□

Theorem 4.5. *All clockable ordinals are countable.*

Proof. We will show that for any ordinal β, if β is clockable, then it is countable. If β is clockable, then there is some natural number p (coding the program P) such that $(\beta+1)_p(0)\downarrow$ (in β steps). We will show that there is a countable elementary substructure S of the universe V in which the sentence $(\beta+1)_p(0)\downarrow$ (in β steps) holds then show by absoluteness that β must be countable.

Let ϕ_p denote the Δ_1 sentence, $(\beta+1)_p(0)\downarrow$ (in β steps). First we note that by the reflection principle (theorem 12.14 of [2]) there exists a countable set S' such that p and all elements of p are in S' and ϕ_p holds in S'.

Let S be the Mostowski Collapse of S' with isomorphism π (Theorem 6.15 of [2]). Since S is isomorphic to S', we know that $S \models (\pi(\beta)+1)_p(0)$ (in $\pi(\beta)$ steps). Since ϕ_p is Δ_1, it is absolute between S and V and so $V \models (\pi(\beta)+1)_p(0)$ (in $\pi(\beta)$ steps). Therefore, since $p(0)$ can only have one halting runtime, $\beta = \pi(\beta)$ and so β is countable. □

This proof can easily be extended to computations with non-zero input.

Definition 4.6. An ordinal α is called *clockable with input x* if there exists some program p such that $(\alpha+1)_p(x)\downarrow$ (after α steps).

Corollary 4.7. *All infinite Turing machine computations (with countable input size) run for countably many steps and thus only use only countably much memory.*

Proof. For any input x, if β is clockable with input x, then β is countable. This can be proved by making only minor adjustments to the proof of Theorem 4.5. First, ϕ_p must be adjusted to say $T(0) = x$ instead of $T(0) =< 0, 0, ... >$. Second, the transitive closure of x must be a member of S' so that when we take the Mostowski Collapse, $\pi(x) = x$. □

This corollary is one of the most important results of this paper as it tells us that all ITM computations can be run a machine with tape size ω_1. A question which is still open is whether there is a countable tape size which is sufficient for all ITM computations. While I have not looked into it yet, I am inclined to say no due to the fact that even on an ω-tape machine, there is a computation of every countable runtime. [1]

References

[1] Joel David Hamkins and Andy Lewis. Infinite time Turing machines. *J. Symbolic Logic*, 65(2):567–604, 2000.

[2] Thomas Jech. *Set Theory*. Springer, Berlin, 2002.

[3] Benjamin Rin. The computational strengths of α-tape infinite time turing machines. *Annals of Pure and Applied Logic*, 165:567–604, 2014.